INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE FÍSICA Y MATEMÁTICAS

Licenciatura en Matemáticas Algorítmicas

Unidad de Aprendizaje:

Análisis Matemático

Conjunto de Cantor

Alumno:

Joel Reyes Garcia

Ciudad de México

27 de marzo de 2026

Cálculo del Conjunto

Para abordar el cálculo del conjunto, decidí programar el motor principal en Rust y consumirlo desde Python. Elegí Rust por el control estricto que ofrece sobre la memoria y su eficiencia pura, algo crucial cuando la cantidad de intervalos crece de forma exponencial con cada iteración. Para no hacer la interconexión un dolor de cabeza, utilicé `maturin` junto con `PyO3`. Estas herramientas me permitieron compilar el código de Rust de forma limpia y usarlo en Python como si fuera un módulo nativo. De esta manera, dejé que Rust hiciera el trabajo duro de cálculo espacial, mientras delegué la lógica de renderizado visual a Python, aprovechando lo práctico que es su ecosistema con `Matplotlib` y `NumPy`.

```rust
use pyo3::prelude::*;

#[pyfunction]
fn conjunto_de_cantor(iteraciones: u32) -> PyResult<Vec<(f64, f64)>> {
    let mut intervalos = vec![(0.0, 1.0)];

    for _ in 0..iteraciones {
        let mut nuevos_intervalos = Vec::with_capacity(intervalos.len() * 2);

        for (inicio, fin) in intervalos {
            let distancia = (fin - inicio) / 3.0;

            nuevos_intervalos.push((inicio, inicio + distancia));
            nuevos_intervalos.push((fin - distancia, fin));
        }
        intervalos = nuevos_intervalos;
    }

    Ok(intervalos)
}

#[pymodule]
fn CantorLib(m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(conjunto_de_cantor, m)?)?;
    Ok(())
}
```

Listing 1: Cálculo del Conjunto de Cantor en Rust usando PyO3

Al diseñar la función `conjunto_de_cantor`, definí el número de iteraciones como un entero sin signo de 32 bits (`u32`), ya que no tiene sentido matemático pedir iteraciones negativas y físicamente es imposible calcular miles de ellas sin saturar la memoria RAM. La función devuelve un vector de tuplas `f64`; usar flotantes de 64 bits era innegociable para no perder precisión computacional cuando los segmentos se vuelven casi microscópicos en las iteraciones más profundas. El algoritmo arranca inicializando la memoria

con el segmento base $[0.0, 1.0]$.

El núcleo del programa es un ciclo que se repite según las iteraciones solicitadas. Un detalle importante de optimización aquí es el uso de `Vec::with_capacity(intervalos.len() * 2)`. Al preasignar la memoria exacta que van a ocupar los subsegmentos en el siguiente nivel, evito que Rust gaste recursos de CPU reasignando memoria dinámica en tiempo de ejecución. En cada paso iterativo, tomo cada intervalo, divido su longitud entre 3 para obtener la `distancia`, y creo los dos nuevos subsegmentos: el izquierdo sumando la distancia al punto de inicio, y el derecho restándola del final.

Para ejemplificar visualmente este comportamiento en memoria, la siguiente figura ilustra cómo el algoritmo toma el segmento original, calcula la distancia de un tercio y empuja las nuevas coordenadas al vector en las primeras iteraciones:

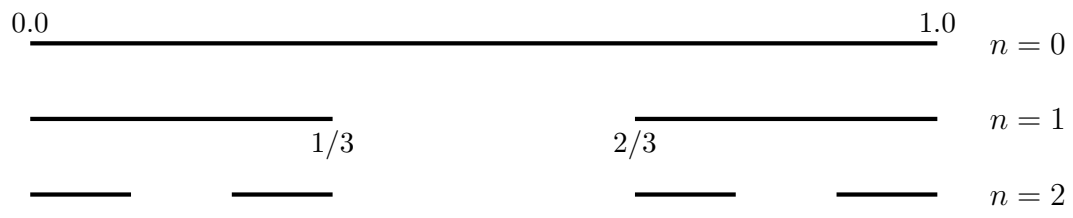

Figura 1: Representación gráfica de la división del segmento en el ciclo de Rust.

Sobreescribo el vector principal con estos nuevos intervalos y, al concluir el cálculo, los datos puros viajan de vuelta a Python listos para graficarse.

Graficación del Conjunto

Una vez construido el motor de cálculo, el siguiente paso fue consumirlo desde el script de Python. El programa compila la librería nativa `CantorLib.so` usando el entorno de `cargo` y la importa dinámicamente mediante `importlib`. Posteriormente, se le solicita al usuario ingresar el número de iteraciones deseadas a través de un `input` estándar.

Para el proceso de graficación, utilicé `Matplotlib` para pintar el lienzo y `NumPy` para manejar los búferes de memoria. El núcleo visual es la función `actualizar_escena`, la cual se conecta a los eventos de la cámara (zoom o paneo) para calcular matemáticamente qué nivel de detalle (LOD) es visible en pantalla y enviar únicamente los segmentos vectoriales necesarios a la tarjeta gráfica, manteniendo la visualización fluida incluso con millones de segmentos calculados.

```python
import os
import sys
import time
import subprocess
import shutil
```

```python
import importlib.util
import math
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

if os.path.exists("CantorLib/Cargo.toml"):
    try:
        subprocess.check_call(
            ["cargo", "build", "--release"],
            cwd="CantorLib", stdout=subprocess.DEVNULL, stderr=subprocess.STDOUT
        )
        shutil.copy("CantorLib/target/release/libCantorLib.so", "CantorLib.so")
    except Exception as e:
        if not os.path.exists("CantorLib.so"): sys.exit(1)

ruta_so = os.path.abspath("CantorLib.so")
spec = importlib.util.spec_from_file_location("CantorLib", ruta_so)
CantorLib = importlib.util.module_from_spec(spec)
sys.modules["CantorLib"] = CantorLib
spec.loader.exec_module(CantorLib)

ITERACIONES = int(input('Elige el numero de iteraciones: \n'))

t0 = time.time()
segs_por_nivel = []
for i in range(ITERACIONES + 1):
    inter = CantorLib.conjunto_de_cantor(i)
    segs_por_nivel.append(np.array(inter, dtype=np.float64))

print(f"Calculados {2**(ITERACIONES+1)-1:,} elementos en {time.time() - t0:.3f}s")

fig, ax = plt.subplots(figsize=(12, 8))
coleccion = LineCollection([], colors='black', linewidths=8.0, antialiased=False)
coleccion.set_capstyle('butt')
ax.add_collection(coleccion)

def actualizar_escena(event_ax=None):
    for txt in list(ax.texts):
        txt.remove()

    xlim = ax.get_xlim()
    ancho_vista = xlim[1] - xlim[0]

    if ancho_vista <= 0: return

    min_dx = ancho_vista / 1920.0

```

```python
    try:
        j_max = int(math.log(1.0 / min_dx) / math.log(3.0)) + 1
    except ValueError:
        j_max = ITERACIONES

    segs_a_dibujar = []

    for i in range(ITERACIONES + 1):
        k = min(i, j_max, ITERACIONES)
        segs_k = segs_por_nivel[k]

        mask = (segs_k[:, 1] >= xlim[0]) & (segs_k[:, 0] <= xlim[1])
        visibles = segs_k[mask]

        if len(visibles) > 0:
            n_vis = len(visibles)
            y_val = -i

            bloque = np.empty((n_vis, 2, 2))
            bloque[:, 0, 0] = visibles[:, 0]
            bloque[:, 0, 1] = y_val
            bloque[:, 1, 0] = visibles[:, 1]
            bloque[:, 1, 1] = y_val

            segs_a_dibujar.append(bloque)

            if i <= j_max:
                for x0, x1 in visibles:
                    dx_real = x1 - x0
                    if dx_real > (ancho_vista * 0.15):
                        caja_inicio = dict(boxstyle="round,pad=0.2", facecolor="#f4f8fa", edgecolor="#0055aa", alpha=0.9)
                        caja_fin = dict(boxstyle="round,pad=0.2", facecolor="#faf4f4", edgecolor="#aa0000", alpha=0.9)

                        ax.annotate(f"{x0:.16g}", xy=(x0, y_val), xytext=(5, 18),
                                    textcoords="offset points", ha='left', va='bottom',
                                    fontsize=9, family='monospace', color='#003366',
                                    bbox=caja_inicio, arrowprops=dict(arrowstyle="->", color="#0055aa", shrinkB=4))

                        ax.annotate(f"{x1:.16g}", xy=(x1, y_val), xytext=(-5, -18),
                                    textcoords="offset points", ha='right', va='top',
                                    fontsize=9, family='monospace', color='#660000',
                                    bbox=caja_fin, arrowprops=dict(arrowstyle="->", color="#aa0000", shrinkB=4))
```

```python

    if segs_a_dibujar:
        coleccion.set_segments(np.vstack(segs_a_dibujar))
    else:
        coleccion.set_segments([])

    if event_ax:
        ax.figure.canvas.draw_idle()

ax.callbacks.connect('xlim_changed', actualizar_escena)

ax.set_xlim(-0.05, 1.05)
ax.set_ylim(-ITERACIONES - 1, 1)
ax.axis('off')
plt.title(f"Conjunto de Cantor - {ITERACIONES} Iteraciones\n",
    fontsize=14)
plt.tight_layout()

actualizar_escena()

plt.show()
```

Listing 2: Implementación de renderizado interactivo en Python (cantor.py)

Resultados y Rendimiento

Al ejecutar el código, la generación del fractal se realiza de manera extraordinariamente rápida. Para un escenario de alta demanda algorítmica, el backend desarrollado en Rust es capaz de calcular $2,097,151$ elementos en tan solo 0.633 segundos. En la Figura 2 se presenta la vista general inicial de la salida, donde la estructura global del Conjunto de Cantor se aprecia de manera satisfactoria.

A simple vista, debido a las limitaciones naturales de renderizado por píxel en una pantalla estándar, `Matplotlib` fusiona visualmente los intervalos más pequeños, causando lo que conocemos como aliasing. Sin embargo, gracias al motor matemático implementado, los datos no se destruyen. Al realizar acercamientos (zoom) sobre la gráfica, el sistema recalcula los vectores visibles y renderiza las coordenadas exactas de forma dinámica. Como se observa en la Figura 3, al navegar hacia las profundidades del fractal, los subsegmentos microscópicos se definen perfectamente y las etiquetas de plano técnico emergen sin ninguna caída en el rendimiento ni pérdida de precisión en los flotantes.

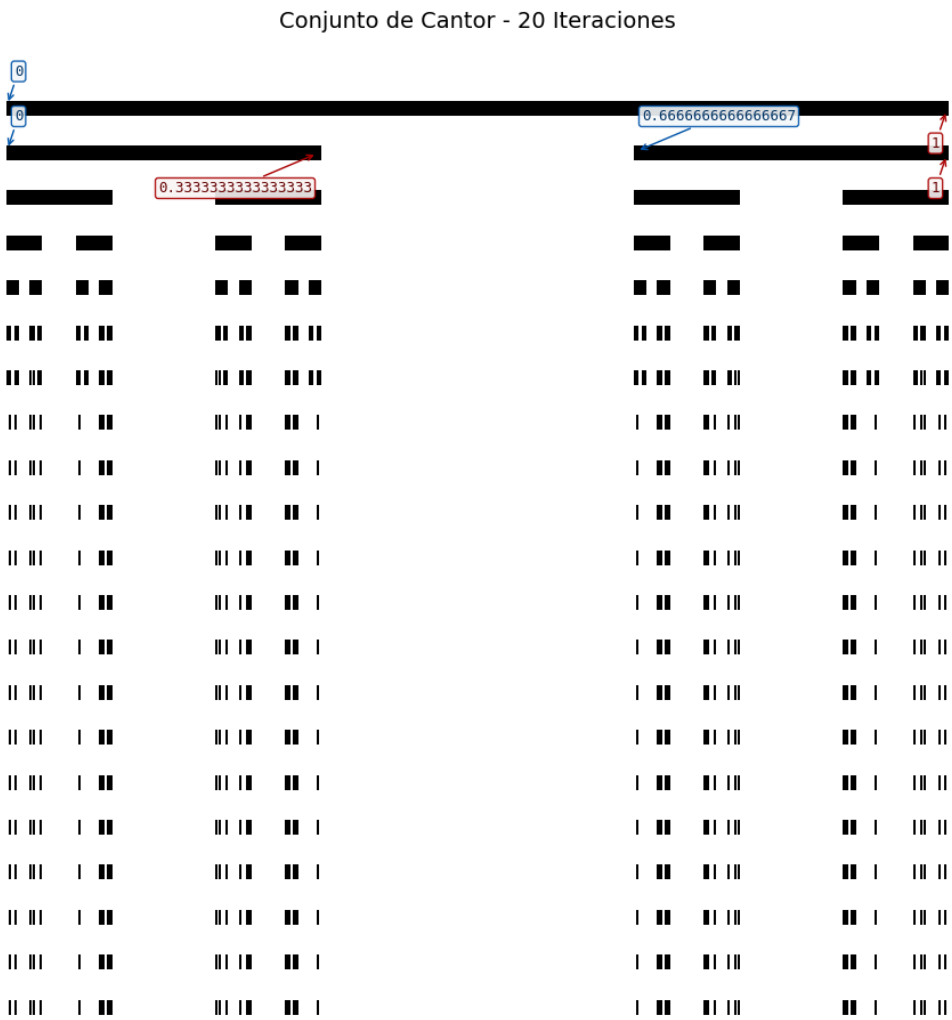

Figura 2: Vista general del Conjunto de Cantor generado. La eficiencia de Rust permite un despliegue inmediato de millones de segmentos.

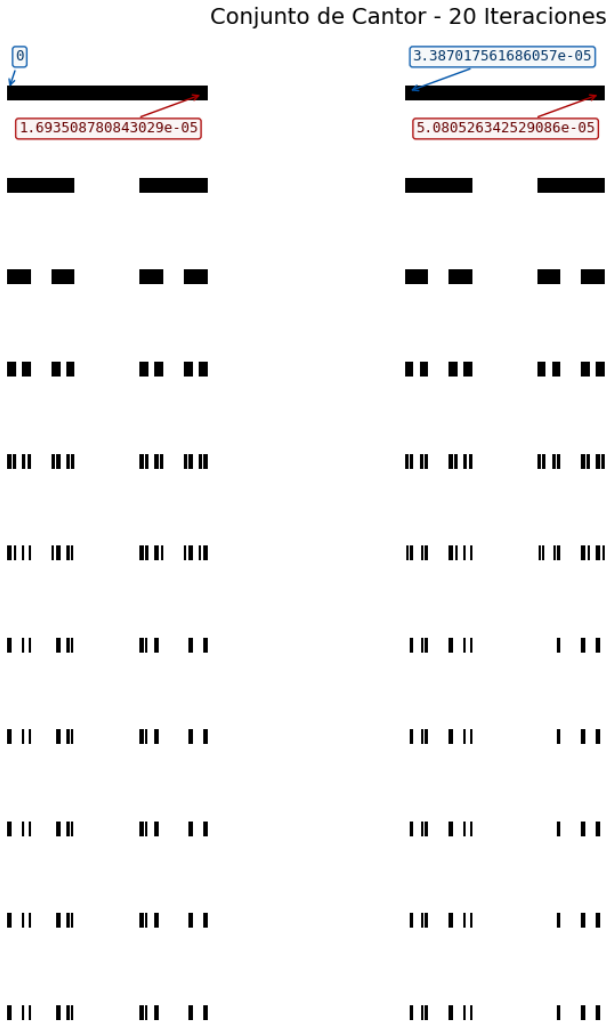

Figura 3: Detalle del fractal al realizar zoom. El motor interactivo revela las coordenadas de alta precisión en tiempo real de los elementos más finos.

Repositorio del Proyecto

El código fuente completo de esta implementación está disponible en GitHub:

https://github.com/Jowl0/Conjunto_De_Cantor